Stage 1.4.2.5.3 — Docling Layout & Reading Order

Document Understanding and Reading Order Integrity for RAG

Objective

We already learned:

Stage 1.4.2.5.1 — Basic OCR

BASIC OCR PIPELINE

```
Image
  ↓
OCR
  ↓
Text
```

Then:

Stage 1.4.2.5.2 — Docling for Scanned PDFs

DOCLING PIPELINE

```
Scanned PDF
     ↓
  Docling
     ↓
DoclingDocument
     ↓
Markdown / structured representation
```

Now we're asking a deeper question:

Can Docling understand the layout of a document and determine the correct logical reading order?

Step 1 — Understand the Problem

Consider a document like this:

VISUAL LAYOUT

```
┌──────────────────────┬──────────────────────┐
│ Heading A            │ Heading B            │
│                      │                      │
│ Paragraph A1         │ Paragraph B1         │
│ Paragraph A2         │ Paragraph B2         │
│                      │                      │
└──────────────────────┴──────────────────────┘
```

A naive text extractor might produce:

NAIVE EXTRACTION

```
Heading A
Heading B
Paragraph A1
Paragraph B1
Paragraph A2
Paragraph B2
```

But the logical reading order might be:

LOGICAL READING ORDER

```
Heading A
Paragraph A1
Paragraph A2

Heading B
Paragraph B1
Paragraph B2
```

That's the problem layout and reading-order analysis tries to solve.

Step 2 — Understand Why This Matters to RAG

This is extremely important. Suppose a PDF contains:

MULTI-COLUMN INPUT

```
Column 1                  Column 2

Azure Event Hubs          Azure Data Explorer

Event ingestion           Analytics

High-volume events        Query processing
```

If the extraction order is wrong, we might create a chunk such as:

CORRUPTED EXTRACTION

```
Azure Event Hubs
Azure Data Explorer
Event ingestion
Analytics
High-volume events
Query processing
```

The text exists, but its relationships have been damaged. That can negatively affect:

RAG DEGRADATION FLOW

```
Chunking
   ↓
Embedding
   ↓
Retrieval
   ↓
LLM context
```

Therefore: Good RAG begins with good document understanding.

Step 3 — Locate the Sample PDF

Download the sample PDF above and place it somewhere accessible from your notebook. For example:

DIRECTORY STRUCTURE

```
rag-learning/
│
├── notebooks/
│
├── data/
│   └── docling_layout_reading_order_sample.pdf
│
└── ...
```

Then:

PYTHON

```
from pathlib import Path

pdf_path = Path(
    "../../data/docling_layout_reading_order_sample.pdf"
)

print(pdf_path.exists())
print(pdf_path)
```

Adjust the path according to where you saved it. We want: True

Step 4 — Check Your Docling Version

We're using your installed: Docling 2.120.3 Let's confirm from the notebook:

PYTHON

```
import importlib.metadata

print(importlib.metadata.version("docling"))
```

Expected: 2.120.3

Step 5 — Create the Document Converter

Use the same approach from Stage 1.4.2.5.2:

PYTHON

```
from docling.document_converter import DocumentConverter

converter = DocumentConverter()
```

Step 6 — Convert the Sample PDF

Run:

PYTHON

```
result = converter.convert(pdf_path)

print(result.status)
```

We want a successful conversion.

Step 7 — Get the DoclingDocument

PYTHON

```
doc = result.document
print(type(doc))
```

You should get a DoclingDocument.

Step 8 — Export to Markdown

This is our first way to observe the result:

PYTHON

```
markdown_text = doc.export_to_markdown()

print(markdown_text)
```

What are we looking for? Look carefully at: Heading order, Paragraph order, Two-column content, Table position, Caption position, Page transitions.

Don't just ask: "Did Docling extract the words?" Ask: "Did Docling preserve the logical structure?" That's the purpose of this stage.

Step 9 — Inspect the Page Structure

Now we want to go deeper than Markdown. The DoclingDocument contains structured information about document elements. Let's first inspect what is available:

PYTHON

```
print(type(doc))
[m for m in dir(doc) if not m.startswith("_")]
```

This is useful because we're working specifically with Docling 2.120.3, rather than blindly copying APIs from another version.

Step 10 — Inspect Document Items

For this stage, one particularly useful concept is the document's items. Try:

PYTHON

```
print(doc.body)
print(doc.__dict__.keys())
```

We're looking for how Docling 2.120.3 represents the document internally. Depending on the exact object structure exposed by your installed version, we'll inspect the relevant collections rather than assuming a particular API.

Step 11 — Why We Are Inspecting the Structure

Imagine Docling internally recognizes something like:

DOCUMENT STRUCTURE

```
Page 1
│
├── Heading
├── Paragraph
├── Paragraph
├── Heading
├── Paragraph
├── Table
└── Caption
```

That's much more useful than: one giant string because later we can make intelligent decisions about chunking. For example:

SEMANTIC CHUNKING

```
Heading
   +
Paragraphs
   ↓
one semantic chunk
rather than blindly doing:
every 500 characters
```

Step 12 — Inspect the Markdown More Carefully

Let's print the first page's extracted Markdown separately if possible, but first simply inspect:

PYTHON

```
print(markdown_text[:5000])
```

Look for something like:

MARKDOWN PREVIEW

```
# Azure Event Processing Architecture

## 1. Overview
...
## 2. Processing Stages
...
| Stage | Component | Purpose |
|---|---|---|
...
```

The exact result will depend on how Docling interprets the generated PDF.

Step 13 — Focus on Reading Order

Our sample contains this logical sequence:

LOGICAL SEQUENCE

```
1. Overview

2. Ingestion Layer

3. Analytics Layer

4. Processing Stages

5. Important Design Considerations
```

We intentionally designed the document so that the visual layout isn't simply a single linear stream. The question we're testing is: Visual position ≠ Logical reading order A document-understanding system needs to infer the latter.

Step 14 — Compare with a Naive Text Extraction Approach

This comparison is useful.

NAIVE PDF EXTRACTION

```
Naive PDF extraction
PDF
 ↓
Text extraction
 ↓
String
Potential problem:
Wrong ordering
Lost relationships
Lost layout
Lost table structure
```

DOCLING EXTRACTION

```
Docling
PDF
 ↓
Document understanding
 ↓
Layout
 ↓
Reading order
 ↓
Structured document
Potentially:
Heading
   ↓
Paragraph
   ↓
Paragraph
   ↓
Table
   ↓
Caption
```

That's why we're learning Docling.

Step 15 — Understand Reading Order in RAG

Suppose our document says: Azure Event Hubs and underneath it: Event ingestion platform. Then in another column: Azure Data Explorer with: Analytics platform. If our parser mixes them up, the resulting embedding might represent a distorted relationship. Instead, we want:

RELATIONSHIPS

```
Azure Event Hubs
    ↓
Event ingestion platform
and:
Azure Data Explorer
    ↓
Analytics platform
```

This produces much better semantic units for subsequent chunking.

Step 16 — Understand the Relationship with Chunking

This is one of the most important lessons from this stage. We previously learned that chunking strategy matters. But now notice:

INGESTION FLOW

```
Document Understanding
        ↓
Layout
        ↓
Reading Order
        ↓
Semantic Structure
        ↓
Chunking
```

Therefore: You shouldn't think of chunking as an isolated operation. The quality of your chunks depends partly on how well you understand the source document. This is why modern RAG ingestion pipelines increasingly use document-understanding frameworks.

Step 17 — Inspect the Table

Our sample PDF contains a table: Stage | Component | Purpose

Stage	Component	Purpose
1	Event Producer	Publishes raw event streaming payload

2	Azure Event Hubs	Ingests high-throughput data streams
3	Azure Data Explorer	Low-latency real-time analytics query engine

After conversion:

PYTHON

```
markdown_text = doc.export_to_markdown()

print(markdown_text)
```

Look for a Markdown table. If Docling preserves it as: | Stage | Component | Purpose | |---|---|---| | 1 | Event Producer | ... | | 2 | Azure Event Hubs | ... | that's an important observation.

Compare that with basic OCR, which could produce:

UNSTRUCTURED OCR OUTPUT

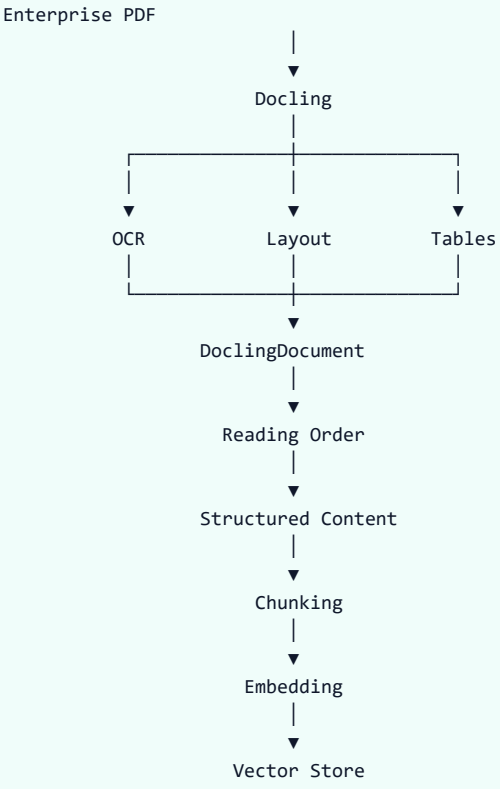
```
Stage Component Purpose
1 Event Producer Publishes...
2 Azure Event Hubs Ingests...
```

The second output contains the words but may have lost the explicit table relationships.

Step 18 — The Architecture We Are Building Toward

Our ingestion architecture is gradually becoming:

INGESTION ARCHITECTURE



This is the conceptual reason we're doing this stage.

Step 19 — What We Are NOT Doing Yet

Don't worry about: OCR engine tuning, Tesseract configuration, image preprocessing, formula enrichment, picture description, multimodal embeddings, table-specific chunking, LangChain integration. Those are separate concerns.

Our question right now is simply: Can Docling correctly understand the spatial organization and logical reading order of our document?

Step 20 — Your Hands-On Experiment

Run these cells in order.

CELL 1

```
from pathlib import Path
from docling.document_converter import DocumentConverter
```

CELL 2

```
pdf_path = Path(
    "path/to/docling_layout_reading_order_sample.pdf"
)

print("Exists:", pdf_path.exists())
```

CELL 3

```
converter = DocumentConverter()
```

CELL 4

```
result = converter.convert(pdf_path)

print("Status:", result.status)
```

CELL 5

```
doc = result.document

print(type(doc))
```

CELL 6

```
markdown_text = doc.export_to_markdown()

print(markdown_text)
```

CELL 7

```
print(doc.__dict__.keys())
```

CELL 8

```
print([m for m in dir(doc) if not m.startswith("_")])
```

The key thing I want you to observe

Don't worry yet about writing a lot of code. After running the conversion, look at the Markdown output and compare it against the visual PDF. We're testing three things:

VERIFICATION CRITERIA

```
1. Did Docling recognize the headings?
        ↓
2. Did it preserve the logical reading order?
        ↓
3. Did it preserve the table structure?
```

Once we see your actual output from Docling 2.120.3, we'll inspect the DoclingDocument structure in the next step and learn how Docling represents layout and reading order internally. That is much more valuable than simply calling an export method and moving on.